DEEP LEARNING - TG CPGET 2026 Revision Handbook

A concept-to-practice guide covering: Introduction to Deep Learning, Neural Networks, Tensors (Scalars/Vectors/Matrices/Higher-D), Tensor Operations, Gradient-Based Optimization & Backpropagation, Neural Network Anatomy, Keras/TensorFlow/Theano/CNTK, Recurrent Neural Networks (LSTM & GRU).

HOW TO USE THIS HANDBOOK

Each major block has: **Concept in Simple Language -> Key Definitions/Formulas -> Examples -> Practice Questions**. Read once for understanding, then use the "Key Definitions" boxes as flash-cards before the exam.

BLOCK 1: INTRODUCTION TO DEEP LEARNING & NEURAL NETWORKS

1.1 What is Deep Learning?

Deep Learning is a subfield of Machine Learning that uses **neural networks with many layers** ("deep" = many layers stacked) to automatically learn hierarchical representations of data, instead of relying on hand-crafted features. Each layer transforms its input into a slightly more abstract, useful representation for the next layer.

1.2 AI vs Machine Learning vs Deep Learning

- **Artificial Intelligence (AI)**: The broad effort to make machines perform tasks that normally require human intelligence.
- **Machine Learning (ML)**: A subset of AI where systems **learn patterns from data** instead of being explicitly programmed with rules.
- **Deep Learning (DL)**: A subset of ML using multi-layered neural networks, especially powerful for unstructured data (images, audio, text).

> **Exam Tip:** Classical ML often needs manual **feature engineering**; Deep Learning learns features automatically from raw data through successive layers.

1.3 Why Deep Learning Works Now (Not Earlier)

1. **Big Data availability** - huge labeled datasets (ImageNet, etc.)
2. **Hardware advances** - GPUs/TPUs enable massive parallel computation.
3. **Algorithmic improvements** - better activation functions, optimizers, regularization (dropout, batch norm).

4. **Open-source frameworks** - TensorFlow, Keras, PyTorch made experimentation accessible.

1.4 What is a Neural Network? (Simple Language)

A neural network is a computational model loosely inspired by the brain's neurons. It consists of layers of interconnected "nodes" (neurons). Each connection has a **weight**; each neuron applies an **activation function** to a weighted sum of its inputs. The network **learns** by adjusting weights to minimize prediction error.

Key Definitions Box

- **Neuron/Unit**: Basic computational node; computes a weighted sum of inputs plus a bias, then applies an activation function.
- **Weight**: A learnable parameter that scales the importance of an input.
- **Bias**: A learnable offset added to the weighted sum.
- **Activation function**: A non-linear function (ReLU, sigmoid, tanh, softmax) applied to a neuron's output, allowing the network to learn non-linear relationships.
- **Feature engineering**: Manually creating input features from raw data (common in classical ML, largely avoided in DL).
- **Representation learning**: Automatically discovering useful features/representations directly from raw data.

1.5 Illustrative Examples

1. **Image classification**: A deep convolutional network learns edges in early layers, shapes in middle layers, and object parts (eyes, wheels) in later layers - all automatically from pixel data.
2. **Feature engineering vs DL**: A classical spam filter might use hand-picked features like "contains word 'free'"; a deep learning model learns useful text features directly from raw email text.
3. **Why GPUs matter**: Training a network with millions of parameters on CPU might take weeks; on GPU (parallel matrix multiplication), the same task takes hours.
4. **Layered abstraction**: A face-recognition network's first layer detects edges, the next detects contours/textures, and deeper layers detect facial parts and finally whole faces.
5. **AI vs ML vs DL nesting**: Self-driving cars use AI (overall goal), which relies on ML (object detection models), which increasingly relies on DL (convolutional neural networks for vision).

1.6 Practice Questions - Block 1

1. Deep Learning primarily differs from classical Machine Learning because it: (a) Requires manual feature engineering (b) Automatically learns hierarchical

representations from raw data (c) Cannot handle unstructured data (d) Does not use layers

2. Which of the following is NOT a reason Deep Learning became practical recently? (a) Availability of large datasets (b) GPU/TPU hardware (c) Decline in data availability (d) Open-source frameworks
3. In a neural network, the parameter that scales the importance of an input signal is called: (a) Bias (b) Weight (c) Activation (d) Gradient
4. A non-linear function applied to a neuron's weighted sum, enabling the network to model complex relationships, is called: (a) Loss function (b) Activation function (c) Optimizer (d) Regularizer
5. Which best describes the relationship between AI, ML, and DL? (a) DL contains ML which contains AI (b) AI contains ML which contains DL (c) They are unrelated fields (d) ML contains AI which contains DL
6. Manually creating input features from raw data before feeding them to a model is called: (a) Representation learning (b) Feature engineering (c) Backpropagation (d) Tensor reshaping
7. In image classification networks, early layers typically learn to detect: (a) Whole objects (b) Edges and simple patterns (c) Class labels (d) Loss values
8. Deep Learning is especially powerful for which type of data? (a) Only small structured tables (b) Unstructured data like images, audio, text (c) Only numeric time-series (d) Only categorical data
9. A "deep" neural network refers to a network with: (a) A single layer (b) Many stacked layers (c) No hidden layers (d) Only output layers
10. The bias term in a neuron: (a) Scales an input's importance (b) Is a learnable offset added to the weighted sum (c) Is the loss function (d) Is always zero

---

BLOCK 2: TENSORS - SCALARS, VECTORS, MATRICES & HIGHER DIMENSIONS

2.1 What is a Tensor?

A tensor is simply a container for numerical data, generalized to any number of dimensions. Tensors are the fundamental data structure used in deep learning (in NumPy, TensorFlow, PyTorch). The number of dimensions (axes) is called the tensor's **rank** or **ndim**.

2.2 Tensor Types by Dimension

Tensor	Rank (ndim)	Description	Example
Scalar (0D tensor)	0	A single number	`x = 5`

Tensor	Rank (ndim)	Description	Example
Vector (1D tensor)	1	An array of numbers (a list)	`[1, 2, 3]`
Matrix (2D tensor)	2	An array of vectors (rows x columns)	`[[1,2],[3,4]]`
3D tensor	3	An array of matrices (e.g., a stack of matrices)	Timeseries batch, RGB image stack
Higher-D tensor (4D, 5D)	4, 5, ...	Arrays of 3D tensors, etc.	Batches of images (4D), batches of video (5D)

2.3 Key Attributes of a Tensor

Every tensor is defined by 3 key attributes:

1. **Number of axes (rank/ndim)**: e.g., a matrix has ndim = 2.
2. **Shape**: A tuple describing the size along each axis, e.g., `(3, 5)` for a matrix with 3 rows, 5 columns. A vector has a shape with a single element, e.g., `(5,)`. A scalar has an empty shape `()`.
3. **Data type (dtype)**: The type of data stored, e.g., `float32`, `int32`, `uint8` (rarely `char`; tensors usually can't store strings directly - text must be pre-processed into numeric tensors).

2.4 Manipulating Tensors in NumPy

- Selecting elements/slices of a tensor is called **tensor slicing**.
- `my_slice = train_images[10:100]` selects samples 10 to 99 (90 samples) - equivalent to `train_images[10:100, :, :]`.
- Negative indices work relative to the end (like Python lists).
- `.shape`, `.ndim`, `.dtype` are the standard NumPy attributes used to inspect a tensor.

2.5 The Notion of Data Batches

Deep learning models typically don't process the entire dataset at once. Data is split into **batches** - small subsets processed together in one forward/backward pass. By convention, the **first axis (axis 0)** of all data tensors in DL is the **sample axis** (or batch axis).

- **Batch size**: Number of samples in one batch (e.g., 128).
- Example: MNIST digit batch of size 128 -> shape `(128, 28, 28)` (128 grayscale 28x28 images).

2.6 Real-World Examples of Data Tensors

Data Type	Typical Tensor Rank	Shape Example
Vector data	2D	`(samples, features)` e.g., (10000, 100)
Timeseries / Sequence data	3D	`(samples, timesteps, features)` e.g., stock prices (250, 390, 3)
Image data	4D	`(samples, height, width, channels)` e.g., (128, 256, 256, 3)
Video data	5D	`(samples, frames, height, width, channels)` e.g., (4, 240, 144, 256, 3)

> **Exam Tip:** Image tensors can follow **channels-last** `(samples, height, width, channels)` (TensorFlow default) or **channels-first** `(samples, channels, height, width)` (Theano convention) - a favorite conceptual MCQ point.

2.7 Illustrative Examples

1. **Scalar**: A single temperature reading, `37.5`, is a 0D tensor.
2. **Vector**: A person's feature vector `[age=25, height=170, weight=65]` is a 1D tensor of shape `(3,)`.
3. **Matrix**: A dataset of 100 people x 3 features is a 2D tensor of shape `(100, 3)`.
4. **3D tensor (timeseries)**: Stock data for 250 trading days, sampled every minute (390 minutes/day), with 3 features (open, high, low) -> shape `(250, 390, 3)`.
5. **4D tensor (images)**: A batch of 128 color images, each 256x256 pixels with 3 color channels -> shape `(128, 256, 256, 3)`.

2.8 Practice Questions - Block 2

1. A single number like `x = 7` is an example of a: (a) 1D tensor (b) 2D tensor (c) 0D tensor / scalar (d) 3D tensor
2. The number of axes of a tensor is also called its: (a) Shape (b) Rank / ndim (c) dtype (d) Batch size
3. A matrix with 4 rows and 6 columns has a shape of: (a) (6, 4) (b) (4, 6) (c) (24,) (d) (4, 6, 1)
4. In deep learning convention, which axis of a data tensor is typically the sample/batch axis? (a) Last axis (b) Middle axis (c) First axis (axis 0) (d) There is no fixed convention

5. Timeseries/sequence data is typically represented as a tensor of shape: (a) (samples, features) (b) (samples, timesteps, features) (c) (samples, height, width) (d) (samples, height, width, channels)
6. A batch of 64 color images of size 128x128 would typically have the shape: (a) (64, 128, 128) (b) (128, 128, 3) (c) (64, 128, 128, 3) (d) (3, 64, 128, 128, 1)
7. Which of the following is NOT one of the three key attributes of a tensor? (a) Number of axes (b) Shape (c) Data type (d) Learning rate
8. Video data is typically represented using a tensor of rank: (a) 3D (b) 4D (c) 5D (d) 2D
9. The "channels-first" convention for image tensors places the channel axis: (a) Last (b) First, right after the batch axis (c) In the middle (d) It does not exist in image tensors
10. If `train_images` has shape (60000, 28, 28), selecting `train_images[10:100]` returns a tensor of shape: (a) (90, 28, 28) (b) (100, 28, 28) (c) (10, 28, 28) (d) (89, 28, 28)

BLOCK 3: TENSOR OPERATIONS

3.1 Element-Wise Operations

Operations applied independently to every entry of a tensor. Examples: `relu(x)` (max(x, 0)), addition of two same-shaped tensors, multiplication.

```
z = x + y       # element-wise addition, x and y must have the same shape
z = relu(x)     # element-wise: max(x, 0) applied to every element
```

These operations are highly parallelizable and are implemented efficiently via vectorized BLAS operations in NumPy (rather than Python for-loops).

3.2 Broadcasting

Broadcasting allows element-wise operations between tensors of **different shapes**, by automatically "stretching" the smaller tensor to match the shape of the larger one, without physically copying data.

Broadcasting Rule (simplified)

1. Axes are added to the smaller tensor to match the ndim of the larger tensor.
2. The smaller tensor is repeated (conceptually) along the new axes to match the larger tensor's shape.

Example: Adding a vector of shape `(10,)` to a matrix of shape `(32, 10)` -> the vector is broadcast across all 32 rows, producing a result of shape `(32, 10)`.

3.3 Tensor Dot (Tensor Product)

The **dot product** (also called tensor product) combines entries in the input tensors, and is

one of the most common, most useful tensor operations - it is NOT element-wise.

- Dot product of two vectors -> a scalar.
- Dot product of a matrix and a vector -> a vector.
- Dot product of two matrices `(a, b)` and `(b, c)` -> a matrix of shape `(a, c)` (the second axis of the first matrix must match the first axis of the second).

```
z = np.dot(x, y)   # or x @ y in modern NumPy
```

> **Key Rule**: For matrix dot product `x.shape = (a, b)` and `y.shape = (b, c)`, output shape is `(a, c)`. The inner dimensions (b) must match.

3.4 Tensor Reshaping

Reshaping means rearranging a tensor's rows/columns into a different shape while keeping the same total number of elements.

```
x.reshape((6, 1))   # reshapes a (3, 2) tensor into a (6, 1) tensor - same 6 total elements
```

A special case is **transposition**: exchanging rows and columns of a matrix, i.e., `x[i, :]` becomes `x[:, i]`.

3.5 Geometric Interpretation of Tensor Operations

Tensors can be interpreted as points/vectors in geometric space:

- **Vector addition**: Geometrically, this is like translating (moving) a point/shape in space by another vector.
- **Dot product**: Related to projections and angles between vectors - relevant to affine transformations.
- Every tensor operation applied by a neural network layer can be viewed as some **geometric transformation** of the input data (rotation, scaling, translation, projection).

3.6 A Geometric Interpretation of Deep Learning

A deep neural network can be understood geometrically as a series of simple geometric transformations (each layer = one transformation) that progressively "uncrumples" a complicated, non-linearly separable data manifold into something that becomes simple (often linearly separable) by the final layer. Think of it like uncrumpling a crumpled paper ball, one careful gesture (layer) at a time, so a complex classification problem becomes simple.

3.7 Illustrative Examples

1. **Element-wise ReLU**: For `x = [-2, 3, -1, 5]`, `relu(x) = [0, 3, 0, 5]` - every element processed independently.
2. **Broadcasting**: Adding a bias vector `b` of shape `(10,)` to a batch of pre-activations of shape `(32, 10)` in a dense layer - `b` is broadcast across all 32 samples.
3. **Tensor dot in a dense layer**: A dense (fully-connected) layer computes `output = relu(dot(W, input) + b)`, where `W` (weights matrix) and `input` are combined via dot product.
4. **Reshaping**: Flattening a `(28, 28)` image into a `(784,)` vector before feeding it to a dense layer, using `reshape((784,))`.
5. **Geometric uncrumpling**: A 2-class spiral dataset that is not linearly separable in 2D can be "unfolded" by a few non-linear layers until a straight line (in the final transformed space) separates the two classes.

3.8 Practice Questions - Block 3

1. Which of the following is an example of an element-wise operation? (a) Matrix multiplication (b) ReLU activation applied to each entry (c) Tensor reshaping (d) Tensor dot product
2. Broadcasting allows element-wise operations between tensors: (a) Only of the exact same shape (b) Of different but compatible shapes, by stretching the smaller one (c) Only between scalars (d) Only along the batch axis
3. If matrix A has shape (5, 3) and matrix B has shape (3, 8), the shape of dot(A, B) is: (a) (5, 8) (b) (3, 3) (c) (8, 5) (d) (5, 3)
4. Rearranging a tensor's elements into a different shape, keeping the same total number of elements, is called: (a) Broadcasting (b) Reshaping (c) Slicing (d) Dot product
5. Exchanging the rows and columns of a matrix is a special case of reshaping called: (a) Flattening (b) Transposition (c) Broadcasting (d) Convolution
6. A dense layer's core computation, `output = relu(dot(W, input) + b)`, combines which two types of tensor operations? (a) Reshaping and slicing (b) Dot product and element-wise operations (with broadcasting for bias) (c) Only broadcasting (d) Only reshaping
7. Geometrically, deep learning can be described as learning a series of transformations that: (a) Randomly shuffle the data with no structure (b) Progressively "uncrumple" complex data manifolds into simpler, separable ones (c) Always reduce data to a single scalar (d) Only rotate data, never scale it
8. Adding a vector of shape (10,) to every row of a matrix of shape (32, 10) is an example of: (a) Tensor dot product (b) Broadcasting (c) Transposition (d) Reshaping
9. For matrix dot product x (shape a,b) dot y (shape b,c), which condition must hold? (a) a must equal c (b) The inner dimensions (b) must match (c) All dimensions must be equal (d) x and y must have the same shape

10. Vectorized tensor operations (via BLAS) are generally preferred over Python for-loops because they are: (a) Slower but more readable (b) Much faster due to parallelization (c) Only usable for 1D tensors (d) Not supported in NumPy

---

BLOCK 4: GRADIENT-BASED OPTIMIZATION & BACKPROPAGATION

4.1 The Learning Problem

Training a neural network means finding the set of weights that minimizes a **loss function** (a measure of how wrong the model's predictions are). This is fundamentally an **optimization problem** - and neural networks are trained using **gradient-based optimization**.

4.2 Derivative of a Tensor Operation

The **derivative** (or gradient, for multi-dimensional inputs) of a function tells us the direction and rate at which the function's output changes as its input changes slightly. For a tensor operation, the **gradient** is the generalization of the derivative to functions that take tensors as input - it's a tensor of the same shape as the input, describing the curvature/slope of the loss function at that point.

Key Definitions Box

- **Loss function (objective function):** Measures the distance between the model's prediction and the true target - the quantity to be minimized during training.
- **Gradient:** The derivative of the loss with respect to the model's weights - indicates the direction of steepest increase; we move in the opposite (negative) direction to reduce loss.
- **Learning rate:** A small scalar controlling how big a step to take along the negative gradient during each weight update.
- **Differentiable:** A function is differentiable if a derivative can be computed at every point; neural network operations (matrix dot, element-wise activations) are chosen to be differentiable.

4.3 Stochastic Gradient Descent (SGD)

Gradient Descent is the process of iteratively moving the weights in the direction that decreases the loss, using the gradient.

Types of Gradient Descent

- **Batch Gradient Descent:** Uses the entire training dataset to compute the gradient before each update - accurate but slow for large data.

- **Stochastic Gradient Descent (SGD):** Uses a single (or a small random "mini-batch" of) sample(s) to estimate the gradient and update weights - noisier but much faster and more scalable; the standard approach in deep learning.
- **Mini-batch SGD:** A compromise - uses small batches (e.g., 32, 64, 128 samples) - the most common approach in practice.

The SGD Update Rule (Formula)

```
weight_new = weight_old - (learning_rate x gradient)
```

- If the learning rate is **too small**, training converges very slowly.
- If the learning rate is **too large**, training may overshoot minima and diverge/oscillate.

Optimizer Variants (built on SGD)

- **SGD with Momentum:** Adds a fraction of the previous update to the current one, helping escape local minima and speeding convergence.
- **RMSprop, Adagrad, Adam:** Adaptive learning-rate optimizers that adjust the learning rate per parameter based on historical gradients - **Adam** is the most commonly used default optimizer in practice.

4.4 Chaining Derivatives: The Backpropagation Algorithm

A neural network is a chain of simple, differentiable tensor operations (layers). To compute how the final loss depends on weights buried deep inside the network, we use the **chain rule of calculus**, applied systematically - this algorithm is called **Backpropagation**.

How Backpropagation Works (Conceptually)

1. **Forward pass:** Input data flows forward through the network, layer by layer, to compute the prediction and then the loss.
2. **Backward pass:** Starting from the loss, gradients are computed layer by layer, moving **backward** through the network, using the chain rule to combine each layer's local gradient with the gradient flowing back from the layer after it.
3. Each weight's gradient is used to update it via the optimizer (e.g., SGD).

Chain Rule (Simplified Formula)

If `y = f(g(x))`, then: `dy/dx = dy/dg * dg/dx` In a network with layers L1 -> L2 -> L3 -> Loss, the gradient of the loss w.r.t. L1's weights is computed by chaining the gradients backward through L3, L2, then L1.

> **Exam Tip:** Backpropagation is essentially "reverse-mode automatic differentiation" applied efficiently to the computational graph of the network - it does NOT require manually deriving formulas for every network architecture; deep learning frameworks (Keras/TensorFlow) compute this automatically.

4.5 Illustrative Examples

1. **Loss function example**: For a house-price prediction model, Mean Squared Error (MSE) measures the average squared difference between predicted and actual prices - training tries to minimize this.
2. **Learning rate too high**: Setting learning rate = 10 for a simple regression may cause the loss to oscillate wildly or diverge instead of converging.
3. **Mini-batch SGD**: Training on ImageNet uses mini-batches of, say, 256 images at a time rather than all 1.2 million images at once, making training feasible on GPU memory.
4. **Momentum analogy**: SGD with momentum behaves like a ball rolling downhill, building up speed in a consistent direction and rolling through small bumps (shallow local minima) instead of getting stuck.
5. **Backpropagation chain**: In a 3-layer network, the gradient for Layer 1's weights depends on the gradient flowing back from Layer 3 through Layer 2 - the chain rule links them together mathematically.

4.6 Practice Questions - Block 4

1. The quantity that a neural network tries to minimize during training is called the: (a) Gradient (b) Loss function (c) Learning rate (d) Activation function
2. The gradient of a function indicates: (a) The exact minimum value (b) The direction and rate of steepest increase of the function (c) The number of layers in a network (d) The batch size
3. In the SGD update rule, weights are updated by moving: (a) In the direction of the gradient (b) In the opposite direction of the gradient, scaled by the learning rate (c) Randomly, ignoring the gradient (d) Only after the entire dataset is seen
4. If the learning rate is set too high during training, the most likely outcome is: (a) Very slow but stable convergence (b) Overshooting and possible divergence/oscillation (c) Guaranteed fastest convergence (d) No effect on training
5. Which optimizer is most commonly used as a strong default choice in modern deep learning? (a) Plain batch gradient descent (b) Adam (c) Linear regression (d) K-means
6. The algorithm that computes gradients of the loss with respect to all weights by applying the chain rule backward through the network is called: (a) Forward propagation (b) Backpropagation (c) Broadcasting (d) Convolution
7. Mini-batch Stochastic Gradient Descent updates weights using: (a) The entire dataset at once (b) A single randomly chosen sample only, always (c) A small random subset (batch) of the training data (d) No data, only random weights
8. SGD with momentum helps primarily by: (a) Removing the need for a learning rate (b) Accumulating past gradients to smooth and speed up convergence (c) Increasing the loss intentionally (d) Preventing forward propagation

9. Neural network operations are chosen to be differentiable primarily because: (a) It makes the code shorter (b) Gradient-based optimization (backpropagation) requires computing derivatives (c) It reduces the number of layers needed (d) It eliminates the need for a loss function
10. In the chain rule dy/dx = dy/dg * dg/dx, this mathematical principle underlies which deep learning algorithm? (a) Batch normalization (b) Backpropagation (c) Dropout (d) Data augmentation

---

BLOCK 5: NEURAL NETWORK ANATOMY & INTRODUCTION TO KERAS/TENSORFLOW

5.1 Anatomy of a Neural Network

Training a neural network revolves around four key components:

1. **Layers**, which are combined into a **Model** (network of layers).
2. The **Loss function**, which defines the feedback signal used for learning.
3. The **Optimizer**, which determines how learning proceeds (updates weights) using the gradient.
4. **Metrics** to monitor during training and testing (e.g., accuracy).

5.2 Layers

A **layer** is a data-processing module that takes one or more tensors as input and outputs one or more tensors. Different layers are appropriate for different tensor formats and types of data processing:

- **Dense (fully-connected) layers**: For simple vector data, shape `(samples, features)`.
- **Recurrent layers (e.g., LSTM)**: For sequence data, shape `(samples, timesteps, features)`.
- **Convolutional layers (Conv2D)**: For image data, shape `(samples, height, width, channels)`. Layers have a **state** (their weights, learned via training) and, when connected together in a compatible way, form a **model**.

Layer Compatibility

"Layer compatibility" refers to the fact that every layer only accepts input tensors of a certain shape and returns output tensors of a certain shape. When stacking layers, output shape of one layer must match the expected input shape of the next.

5.3 Models (Networks of Layers)

A deep learning **model** is a directed, acyclic graph of layers. The most common is a linear stack of layers (Sequential model), but more complex topologies exist:

- **Two-branch networks**
- **Multi-head networks**
- **Inception blocks / residual connections** Choosing the right network architecture (topology) is more art than science, though certain patterns are well-established for certain data types (e.g., CNNs for images, RNNs for sequences).

5.4 Loss Functions and Optimizers

Once the network architecture is defined, two more choices are needed:

- **Loss function**: The quantity minimized during training; must be chosen to match the task:
  - **Binary crossentropy** - for two-class (binary) classification problems.
  - **Categorical crossentropy** - for multi-class classification problems.
  - **Mean Squared Error (MSE)** - for regression problems.
- **Optimizer**: Determines exactly how the network's weights will be updated based on the loss's gradient (e.g., SGD, RMSprop, Adam).

> **Exam Tip:** Using the wrong loss function for a task (e.g., MSE for a classification problem) is a common mistake highlighted in DL syllabi - always match the loss to the task type.

5.5 Introduction to Keras and Deep Learning Frameworks

Keras is a high-level, user-friendly deep learning API written in Python that allows easy and fast prototyping of neural networks. Keras itself does not implement low-level tensor operations - it relies on a **backend engine** to do so.

Backend Engines Supported Historically by Keras

Framework	Developed By	Notes
TensorFlow	Google	Most widely used backend; became the default/standard backend for Keras (now Keras is tightly integrated into TensorFlow as `tf.keras`)
Theano	University of Montreal (MILA)	One of the earliest DL libraries; development discontinued
CNTK	Microsoft (Cognitive Toolkit)	Microsoft's deep learning framework; no longer actively developed

Keras provides a consistent, high-level API regardless of which backend performs the actual tensor computations - this abstraction lets developers write backend-agnostic code

(historically).

5.6 Illustrative Examples

1. **Dense layer for vector data**: A model classifying iris flowers from 4 numeric features uses Dense layers, since input is simple vector data `(samples, 4)`.
2. **Choosing loss function**: A binary spam/not-spam classifier uses `binary_crossentropy`; a 10-digit MNIST classifier uses `categorical_crossentropy`; a house-price regressor uses `mse`.
3. **Layer compatibility**: A Conv2D layer outputting shape `(None, 26, 26, 32)` cannot be directly followed by a Dense layer without first using a `Flatten()` layer to reshape it into a vector.
4. **Sequential model**: Stacking `Dense(32) -> Dense(16) -> Dense(1)` layers in order to build a simple regression model in Keras.
5. **Backend abstraction (historical)**: The same Keras code for building a CNN could historically be run with either the TensorFlow or Theano backend with minimal changes, since Keras abstracted away the low-level operations.

5.7 Practice Questions - Block 5

1. Which of the following is NOT one of the 4 key components in the anatomy of a neural network? (a) Layers (b) Loss function (c) Optimizer (d) Database schema
2. A layer suitable for processing simple vector data of shape (samples, features) is a: (a) Convolutional layer (b) Dense (fully-connected) layer (c) Recurrent layer (d) Embedding layer only
3. For a multi-class classification problem (more than 2 classes), the appropriate loss function is typically: (a) Mean Squared Error (b) Binary crossentropy (c) Categorical crossentropy (d) Mean Absolute Error
4. Keras is best described as: (a) A low-level tensor computation engine (b) A high-level API that relies on a backend engine for tensor operations (c) A database management system (d) A hardware accelerator
5. Which of the following was developed by Google and became the dominant backend for Keras? (a) Theano (b) CNTK (c) TensorFlow (d) Scikit-learn
6. "Layer compatibility" refers to the requirement that: (a) All layers must have the same number of neurons (b) A layer's output shape must match the next layer's expected input shape (c) All layers must use the same activation function (d) Layers must be trained separately
7. A Sequential model in Keras refers to: (a) A model with multiple independent branches only (b) A linear stack of layers (c) A model with no layers (d) A convolutional-only architecture
8. Which loss function is most appropriate for a regression task (predicting a continuous value)? (a) Categorical crossentropy (b) Binary crossentropy (c) Mean Squared Error (d) Softmax

9. Which framework among the following was developed by the University of Montreal and later discontinued? (a) TensorFlow (b) CNTK (c) Theano (d) PyTorch
10. To connect a Conv2D layer's output to a Dense layer, which operation is typically needed first? (a) Broadcasting (b) Flatten (reshape into a vector) (c) Backpropagation (d) Dropout

---

BLOCK 6: RECURRENT NEURAL NETWORKS - LSTM AND GRU

6.1 Why Recurrent Neural Networks (RNNs)?

Dense and convolutional networks have **no memory** - each input is processed independently, with no information about previous inputs. For sequence data (text, timeseries, speech), order and context matter. A **Recurrent Neural Network (RNN)** processes sequences by iterating through timesteps while maintaining an internal **state** that carries information from previous timesteps forward.

6.2 The Basic Recurrent Layer (SimpleRNN)

At each timestep `t`, a recurrent layer:

1. Takes the current input `x_t` and the previous state `state_{t-1}`.
2. Combines them (via weight matrices and an activation function) to produce a new output and updated state.

```
output_t = activation(dot(W, x_t) + dot(U, state_t-1) + b)
state_t = output_t     # this output becomes the state for the next timestep
```

Limitation: The Vanishing Gradient Problem

SimpleRNN struggles to learn **long-term dependencies** - as gradients are backpropagated through many timesteps, they tend to shrink toward zero (vanish), making it very hard for the network to learn relationships between distant timesteps.

6.3 LSTM (Long Short-Term Memory)

LSTM is a special kind of recurrent layer, designed specifically to address the vanishing-gradient/long-term-dependency problem. It adds a **carry track** (cell state) that runs through the sequence, allowing information to be preserved over many timesteps, regulated by learnable **gates**.

LSTM Gates (Conceptual, Simple Language)

- **Forget gate**: Decides what information to discard from the cell state.

- **Input gate**: Decides what new information to add/store into the cell state.
- **Output gate**: Decides what part of the cell state to output as the layer's output at this timestep.
- **Cell state (carry)**: The "conveyor belt" that carries relevant long-term information across timesteps, modified only lightly by the gates at each step.

> **Analogy:** Think of the LSTM's cell state as a conveyor belt carrying information down the sequence; gates act like valves that let information in, keep it, or let it out, protecting important information from being lost or overwritten too quickly.

6.4 GRU (Gated Recurrent Unit)

GRU is a simplified, more computationally efficient alternative to LSTM, also designed to fight the vanishing-gradient problem, but with **fewer gates** (typically an update gate and a reset gate) and no separate cell state (it merges the cell state and hidden state).

LSTM vs GRU

Feature	LSTM	GRU
Number of gates	3 (forget, input, output)	2 (update, reset)
Separate cell state	Yes	No (merged with hidden state)
Parameters	More	Fewer
Training speed	Slower	Faster (fewer parameters)
Performance	Often slightly better on complex/long sequences	Often comparable, more efficient on simpler tasks

6.5 A Recurrent Layer in Keras

In Keras, recurrent layers (`SimpleRNN`, `LSTM`, `GRU`) expect input of shape `(batch_size, timesteps, input_features)`. By default, a recurrent layer in Keras returns only the output of the last timestep (a 2D tensor); setting `return_sequences=True` makes it return the full sequence of outputs at every timestep (a 3D tensor) - necessary when stacking multiple recurrent layers.

Key Definitions Box

- **Timestep**: One "step" of a sequence (e.g., one word in a sentence, one day of stock prices).
- **Hidden state**: The internal memory vector carried forward across timesteps.
- **return_sequences**: A Keras parameter controlling whether a recurrent layer outputs at every timestep or just the last one.

- **Bidirectional RNN**: Processes the sequence both forward and backward, often improving performance by using both past and future context.

6.6 Illustrative Examples

1. **Sentiment analysis with LSTM**: An LSTM reads a movie review word by word, maintaining context (e.g., remembering "not" earlier in the sentence) to correctly classify sentiment even when negation appears far from the key sentiment word.
2. **Stock price prediction**: A GRU-based model processes 30 days of stock features to predict the next day's price, using its gates to retain relevant recent trends while forgetting stale, irrelevant fluctuations.
3. **Stacking recurrent layers**: Using `LSTM(32, return_sequences=True)` followed by another `LSTM(32)` requires `return_sequences=True` on the first layer so it passes a full sequence, not just a single final output, to the second layer.
4. **Vanishing gradient illustration**: A SimpleRNN trying to connect a subject at the start of a long paragraph to a pronoun at the end often fails, because gradient signals shrink drastically over many timesteps - LSTM/GRU largely fix this.
5. **Bidirectional LSTM**: For named-entity recognition in text, a Bidirectional LSTM considers both preceding and following words in a sentence to more accurately tag an ambiguous word.

6.7 Practice Questions - Block 6

1. The key advantage of a Recurrent Neural Network over a Dense network for sequence data is that RNNs: (a) Have no memory (b) Maintain an internal state that carries information across timesteps (c) Cannot process text (d) Require no training
2. The main problem SimpleRNN suffers from when learning long sequences is: (a) Overfitting only (b) The vanishing gradient problem (c) Too few parameters (d) Lack of activation functions
3. LSTM addresses the vanishing gradient / long-term dependency problem primarily by introducing: (a) More Dense layers (b) A cell state (carry track) regulated by gates (c) A larger learning rate (d) Convolution operations
4. Which of the following is NOT one of the three main gates in a standard LSTM? (a) Forget gate (b) Input gate (c) Output gate (d) Convolution gate
5. Compared to LSTM, GRU is generally characterized by: (a) More gates and more parameters (b) Fewer gates, no separate cell state, and fewer parameters (c) No gates at all (d) Only being usable for images
6. In Keras, the parameter that makes a recurrent layer output values at every timestep (rather than just the last one) is: (a) return_state (b) return_sequences=True (c) stateful=True (d) go_backwards=True
7. The expected input shape for a recurrent layer in Keras is: (a) (samples, features) (b) (samples, height, width, channels) (c) (batch_size, timesteps, input_features) (d) (batch_size, channels)

8. A Bidirectional RNN improves performance by: (a) Ignoring past context entirely (b) Processing the sequence in both forward and backward directions (c) Removing gates (d) Only working on images
9. When stacking two LSTM layers, the first LSTM layer typically needs which setting? (a) return_sequences=True (b) return_sequences=False (c) No special setting is needed (d) activation=None
10. GRU's typical gates are: (a) Forget and output gates (b) Update and reset gates (c) Input and forget gates (d) Convolution and pooling gates

---

QUICK-REVISION FORMULA & FACT SHEET

Item	Value/Fact
Scalar / Vector / Matrix ranks	0D / 1D / 2D tensors respectively
Image data tensor shape (channels-last)	(samples, height, width, channels) - 4D
Video data tensor shape	(samples, frames, height, width, channels) - 5D
Timeseries data tensor shape	(samples, timesteps, features) - 3D
Matrix dot product rule	(a,b) dot (b,c) -> (a,c); inner dims must match
SGD update rule	weight_new = weight_old - (learning_rate x gradient)
Chain rule	dy/dx = dy/dg * dg/dx (basis of backpropagation)
Binary classification loss	binary_crossentropy
Multi-class classification loss	categorical_crossentropy
Regression loss	mean_squared_error (MSE)
Keras historical backends	TensorFlow, Theano, CNTK
LSTM gates	Forget, Input, Output (+ cell state)
GRU gates	Update, Reset (no separate cell state)
Keras flag to output full sequence	return_sequences=True

ANSWER KEYS

Block 1: 1-b, 2-c, 3-b, 4-b, 5-b, 6-b, 7-b, 8-b, 9-b, 10-b **Block 2:** 1-c, 2-b, 3-b, 4-c, 5-b, 6-c, 7-d, 8-c,

9-b, 10-a **Block 3:** 1-b, 2-b, 3-a, 4-b, 5-b, 6-b, 7-b, 8-b, 9-b, 10-b **Block 4:** 1-b, 2-b, 3-b, 4-b, 5-b, 6-b, 7-c, 8-b, 9-b, 10-b **Block 5:** 1-d, 2-b, 3-c, 4-b, 5-c, 6-b, 7-b, 8-c, 9-c, 10-b **Block 6:** 1-b, 2-b, 3-b, 4-d, 5-b, 6-b, 7-c, 8-b, 9-a, 10-b

---

FINAL REVISION CHECKLIST

- [ ] Can you define scalar, vector, matrix, and 3D+ tensors, and identify their ndim/shape?
- [ ] Can you list the tensor shape convention for vector, timeseries, image, and video data?
- [ ] Can you explain broadcasting and tensor dot product with a shape-matching example?
- [ ] Can you state the SGD update rule and explain effects of learning rate too high/low?
- [ ] Can you explain backpropagation using the chain rule in your own words?
- [ ] Can you list the 4 anatomy components of a neural network (layers, model, loss, optimizer)?
- [ ] Can you match loss functions to task types (binary/multi-class/regression)?
- [ ] Can you name Keras's historical backend engines and who developed each?
- [ ] Can you explain why SimpleRNN struggles with long sequences, and how LSTM/GRU fix it?
- [ ] Can you differentiate LSTM vs GRU gates and explain return_sequences in Keras?